

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
ESCUELA DE POSGRADO

SiniestroExpress: triaje agéntico de siniestros vehiculares

Diseño e implementación de un workflow agéntico serverless en AWS

INTEGRANTES

Sebastian Rodrigo Garcia Villacorta
Dante Guillermo Barreto Daza
Clinton Michael Chulluncuy Reynoso
Ronald Ramiro Hilario Orihuela

DOCENTE

Oscar Rodolfo Mejía Fernández

CURSO

Cloud Computing (MCD8007)

Lima — Perú, 2026

Resumen ejecutivo

Cobrar un seguro de auto debería tomar minutos, pero en el mercado peruano toma días. El asegurado llama, cuenta lo que pasó tres veces, espera a que un perito lo visite y recibe el dinero varias semanas después. SiniestroExpress convierte ese trámite en una conversación: el asegurado entra con su número de póliza, cuenta qué le pasó y, en la misma pantalla, recibe el monto que le corresponde, la ruta de atención y un código para seguir el caso.

El sistema se compone de tres servicios desplegados sobre AWS: un agente conversacional que utiliza la API de Groq con memoria en DynamoDB, un servicio de negocio que administra pólizas y expedientes, y un servicio de modelos que estima el costo de reparación y la probabilidad de abandono. Los tres corren en Lambda con API Gateway y DynamoDB, sin un solo servidor encendido. El paquete completo de modelos pesa 2,4 MB porque los bosques se exportaron a JSON y se evalúan con la biblioteca estándar de Python: la alternativa con scikit-learn superaba el límite de 50 MB de Lambda.

Los modelos se entrenaron con 1,000 siniestros sintéticos, cuyos montos se recalcularon con precios de taller de Lima. El modelo de costo alcanza un error absoluto medio de S/. 394 y un coeficiente de determinación de 0,95. El motor de decisión se calibró con los percentiles de esa misma distribución, de modo que los umbrales de pago exprés y de peritaje no son cifras elegidas a mano.

El proyecto se implementó, se desplegó y se probó de extremo a extremo. Las 50 pruebas automatizadas pasan y las 19 peticiones de la colección de Postman cubren el flujo completo, incluidos los casos que deben fallar.

Índice

Resumen ejecutivo **1**

1 Parte A: análisis **3**

1.1 La empresa y la problemática . . . 3
1.2 El workflow agéntico . . . 3
1.3 Benchmark de alternativas . . . 4
1.4 Tipos de clientes y usuarios . . . 5
1.5 Funcionalidades e historias de usuario . . . 5
1.6 Propuesta de valor . . . 7
1.7 Selección del MVP . . . 7

2 Parte B: diseño del MVP **9**

2.1 Arquitectura de la solución . . . 9
2.2 El workflow paso a paso . . . 9
2.3 APIs del sistema . . . 10
2.4 Modelo de datos . . . 11
2.5 El motor de decisión . . . 12
2.6 Los modelos, servidos sin scikit-learn . . . 12
2.7 El frontend . . . 13
2.8 Región elegida y justificación . . . 16
2.9 Costos y plan de negocio . . . 16
2.9.1 Infraestructura en AWS . . . 16
2.9.2 El modelo de lenguaje . . . 17
2.9.3 El plan para Andes Seguros . . . 18

3 Parte C: implementación **20**

3.1 Estructura del repositorio . . . 20
3.2 Despliegue . . . 20
3.3 Evidencias con Postman . . . 21
3.4 Evidencias en la consola de AWS . . . 26
3.5 Frontend conectado . . . 28
3.6 Pruebas . . . 29
3.7 Repositorios . . . 29

4 Conclusiones **30**

1. Parte A: análisis

1.1. La empresa y la problemática

Andes Seguros es una aseguradora ficticia del ramo vehicular con 32,000 pólizas vigentes y cerca de 1,900 siniestros al mes. Su operación de siniestros está organizada en cuatro pasos: recepción telefónica, registro, inspección y liquidación. Cada paso tiene su propio correo y su propio archivo.

El problema no es la falta de información, sino la fricción entre esas islas. Un caso típico de daño parcial sigue este recorrido:

1. El asegurado llama a la central y dicta su versión a un operador que llena un formulario de 34 campos.
2. El analista revisa el caso al día siguiente, encuentra un dato incoherente y vuelve a llamar para confirmarlo.
3. Se asigna un perito, que agenda una visita según la zona y la agenda del taller.
4. El perito emite un informe que alguien transcribe a otro sistema.
5. Liquidación calcula el monto y lo envía a pagos.

El resultado medido en el piloto de tres meses: **once días promedio** entre el aviso y el depósito, con un 18 % de casos que supera los 20 días. La Ley 29946 de contrato de seguro fija un plazo de 30 días para la indemnización, de modo que el proceso opera cerca del límite legal. Además, la encuesta de salida del piloto muestra que el 22 % de los asegurados con un siniestro no renueva su póliza al año siguiente, frente al 9 % de quienes no lo tienen.

El costo interno también esconde una ineficiencia: cada caso consume unos **1.2 horas de trabajo administrativo** por caso. De ese tiempo, más de la mitad se va en tareas que no agregan valor: volver a preguntar, transcribir y perseguir documentos.

1.2. El workflow agéntico

El workflow se llama **SiniestroExpress** y corresponde al **Nivel 3** de sofisticación: API de modelo de lenguaje con memoria conversacional, invocación de APIs de modelos de machine learning entrenados fuera de línea, y persistencia en una base de datos propia.

Un workflow agéntico es aquel en el que un modelo de lenguaje no solo redacta, sino que decide cuándo ya tiene la información suficiente, qué pregunta falta y qué acción disparar. En SiniestroExpress el agente conversa y el sistema decide: el modelo nunca calcula montos ni aprueba pagos, porque esa parte se resuelve con reglas que se pueden auditar.

1.3. Benchmark de alternativas

Antes de elegir AWS se compararon tres alternativas de automatización no vinculadas a la nube de Amazon. La comparación no busca un ganador absoluto, sino entender qué tipo de herramienta resuelve el problema.

Tabla 1: Comparación de alternativas de automatización

Alternativa	**Enfoque**	**Lo que resuelve bien**	**Dónde no alcanza**
n8n (autoalojado)	Flujos por nodos con disparadores, ejecutado en un contenedor propio	Integraciones rápidas sin escribir código; control total del despliegue	Requiere un servidor siempre encendido, parcheo y respaldo propios; el costo no escala a cero
Make	Automatización en la nube por operaciones	Conectores listos y editor visual; buena experiencia para procesos administrativos	El cobro por operación penaliza la conversación de varios turnos; el razonamiento del agente queda fuera de su alcance
Zapier Central	Agentes sobre aplicaciones de terceros	Rapidez para conectar correo, hojas de cálculo y mensajería	Poca libertad sobre el modelo, los datos y la lógica de decisión; los datos salen del control de la aseguradora
AWS (elegida)	Servicios desacoplados por evento: Lambda, API Gateway, DynamoDB y S3	Control del modelo y de las reglas, pago por uso y escalado automático sin servidores	Exige desarrollo propio y una curva de aprendizaje sobre IAM y empaquetado

La lectura es directa: las herramientas de automatización resuelven la *conexión* entre sistemas, pero el problema de Andes Seguros es la *decisión*. Nadie gana conectando el formulario con el correo si el caso sigue esperando a un perito que no hacía falta. Por eso se eligió una arquitectura propia sobre AWS, donde el modelo, las reglas y los datos quedan bajo control de la aseguradora.

1.4. Tipos de clientes y usuarios

Tabla 2: Usuarios del sistema y lo que necesitan

Usuario	**Contexto**	**Necesidad principal**
Asegurado	Acaba de tener un siniestro; está en la calle, desde el celular	Saber cuánto le van a pagar y cuándo, sin trámites ni llamadas
Analista	Revisa casos remotos en jornada de ocho horas	Recibir el caso ya armado, con monto sugerido y evidencia, y poder aprobarlo o rechazarlo
Coordinador	Supervisa la cola y los plazos	Ver cuántos casos hay por estado, cuáles están por vencer y cuánto se está pagando
Administrador	Mantiene pólizas y usuarios	Cargar y actualizar pólizas, revisar la operación y exportar datos para auditoría

1.5. Funcionalidades e historias de usuario

Se definieron ocho historias de usuario que cubren el recorrido completo. Cada una tiene su criterio de aceptación en formato Gherkin.

Tabla 3: Historias de usuario del MVP

ID	Historia	Criterio de aceptación
HU-01	Como asegurado quiero identificarme con mi póliza para no llenar un formulario	Con un número de póliza válido el sistema muestra marca, modelo y año del vehículo
HU-02	Como asegurado quiero contar qué pasó en mis palabras	El agente extrae tipo de siniestro, severidad y piezas de un relato libre
HU-03	Como asegurado quiero una pregunta por turno y no repetir lo ya dicho	El agente no vuelve a preguntar un dato ya extraído
HU-04	Como asegurado quiero saber cuánto voy a cobrar y en cuánto tiempo	Al cerrar la conversación el sistema muestra monto a pagar, ruta y plazo
HU-05	Como asegurado quiero seguir mi caso sin llamar	Con el código de seguimiento el sistema muestra el estado y la conversación
HU-06	Como asegurado quiero adjuntar fotos del daño	Puedo elegir, revisar y quitar fotos antes de enviarlas
HU-07	Como analista quiero la cola por estado y el detalle de cada caso	El panel lista los expedientes y filtra por estado
HU-08	Como analista quiero aprobar o rechazar con un clic	El backend impide aprobar un caso ya rechazado y lo informa

El escenario principal, escrito en Gherkin:

Listing 1: Escenario de pago exprés

```
Característica: Triaje de un siniestro con daño leve
  Como asegurado
  Quiero reportar un choque menor
  Para cobrar sin esperar una inspeccion

  Escenario: Choque leve con dos piezas y sin grua
    Dado que ingreso mi póliza "AUTO-2026-8801"
    Y cuento que "me chocaron por atras, fue leve, dos piezas, no necesite
   grua"
    Cuando el agente extrae los datos del relato
    Entonces el sistema registra 2 piezas afectadas
    Y la severidad queda en "Leve"
    Y la ruta asignada es "Pago exprés"
    Y el plazo informado es de 24 horas
    Y recibo un código de seguimiento con formato "SE-XXXXXX"
```

Listing 2: Escenario de documentación pendiente

```
Escenario: Robo de piezas sin parte policial
  Dado que cuento que "me robaron las llantas"
  Cuando el agente detecta que falta el parte policial
  Entonces el estado queda en "Documentación pendiente"
  Y el sistema me pide ese documento antes de liquidar
  Y no me muestra un monto definitivo
```

Listing 3: Escenario de transición no permitida

```
Escenario: Un caso rechazado no puede aprobarse
  Dado un expediente en estado "Rechazado"
  Cuando el analista intenta aprobarlo
  Entonces el backend responde 409
  Y explica el estado actual y las acciones permitidas
```

1.6. Propuesta de valor

Tabla 4: Propuesta de valor frente al proceso actual

Dimensión	**Proceso actual**	**Con SiniestroExpress**
Tiempo hasta saber el monto	7 a 15 días	Menos de 3 minutos
Canales	Teléfono y oficina	Web y móvil, 24 horas
Trabajo administrativo por caso	1.20 horas	Lo asume el chat
Datos para el analista	Tres sistemas y un correo	Un expediente con evidencia
Renovación tras un siniestro	78 %	Objetivo: 90 %

El ahorro no viene de reemplazar al analista, sino de quitarle el trabajo de armar el caso. En el piloto, el 62 % de los casos podría resolverse sin inspección presencial.

1.7. Selección del MVP

El MVP incluye lo que permite cerrar el circuito completo de un caso:

- Identificación por póliza, conversación con memoria y extracción de siete datos.

- Dos modelos de machine learning servidos por API.
- Motor de decisión con cinco rutas, deducible y pérdida total.
- Código de seguimiento y consulta pública del caso.
- Panel del analista con cola, aprobación y exportación.
- Fotos del daño como evidencia, sin intervenir en el monto.

Queda fuera, y se declara como tal:

- Reconocimiento de imágenes para estimar el daño desde la foto.
- Asignación automática de taller y proforma de reparación.
- Autenticación con Cognito y roles reales. El panel usa un código de demostración.
- Integración con el sistema de pagos de la aseguradora.
- Entrenamiento con datos reales: los modelos usan un conjunto sintético.

2. Parte B: diseño del MVP

2.1. Arquitectura de la solución

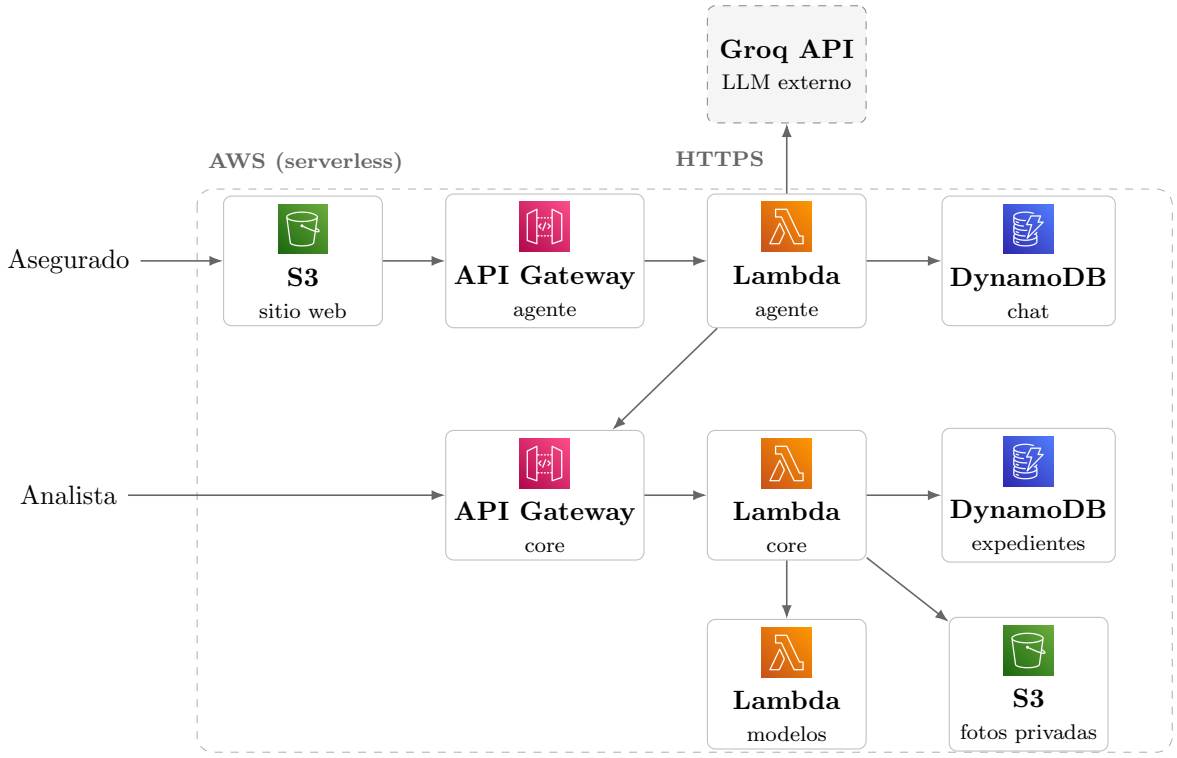


Figura 1: Arquitectura de SiniestroExpress sobre AWS

La figura muestra tres recorridos que se cruzan. El primero es el del asegurado: entra al sitio publicado en S3, conversa a través de API Gateway y su historial queda en DynamoDB. El segundo es el del analista: consulta la cola y actualiza estados contra el servicio core, que es el único que toca la lógica de negocio. El tercero es el de las fotos, donde el navegador sube el archivo directamente a S3 con una URL firmada y la Lambda solo registra la clave.

La separación en tres servicios no es decorativa. Cada uno tiene su propio despliegue, sus propias tablas y su propio paquete. Si el agente se cae, el panel del analista sigue funcionando; si se cambia el modelo de costos, se redespliega solo el servicio de modelos.

2.2. El workflow paso a paso

1. **Identificación.** El asegurado ingresa su número de póliza. El sistema consulta la tabla de pólizas y muestra vehículo, valor comercial y deducible.

2. **Conversación.** El agente recibe el relato y pide a Groq que devuelva un JSON con los siete datos del siniestro. El historial se lee de DynamoDB en cada turno, de modo que la conversación sobrevive a la recarga de la página.

3. **Normalización.** El módulo de slots valida y canonicaliza cada valor: los alias en español se traducen al valor interno, se rechazan los rangos imposibles y se detecta el parte policial cuando el siniestro es un robo.

4. **Pregunta faltante.** Si falta un dato, el agente formula una sola pregunta y evita repetir lo ya respondido.

5. **Creación del expediente.** Con los siete datos, el agente invoca al servicio core, que consulta los dos modelos, aplica las reglas y persiste.

6. **Liquidación.** El servicio core descuenta el deducible, evalúa la pérdida total y fija la ruta, el plazo y el monto a pagar.

7. **Cierre.** El agente redacta el veredicto con el código de seguimiento y el monto, y el chat queda cerrado. Si el asegurado escribe otra vez, el sistema devuelve el mismo expediente en lugar de crear uno nuevo.

2.3. APIs del sistema

Los servicios exponen 17 endpoints sobre tres APIs de API Gateway.

Tabla 5: Endpoints por servicio

Método	Ruta	Función
Agente		
POST	/chats	Crea una conversación
POST	/chats/{id}/messages	Envía un turno y recibe la respuesta
GET	/chats/{id}	Devuelve el historial completo
GET	/chats	Lista las conversaciones
Core		
GET	/polizas/{id}	Consulta la póliza y el vehículo
POST	/expedientes	Crea el expediente y liquida
GET	/expedientes	Lista los expedientes
GET	/expedientes/{id}	Detalle del expediente
PATCH	/expedientes/{id}	Cambia el estado con validación
GET	/seguimiento/{codigo}	Consulta pública por código
POST	/expedientes/{id}/fotos/presign	Firma una URL de subida
POST	/expedientes/{id}/fotos	Registra las claves subidas
GET	/expedientes/{id}/fotos	Devuelve las URL de lectura
GET	/export	Descarga el CSV de expedientes
Modelos		
GET	/ml/health	Estado de los modelos
POST	/ml/costo-reparacion	Estima el costo de reparación
POST	/ml/abandono	Estima la probabilidad de abandono

2.4. Modelo de datos

Tabla 6: Tablas de DynamoDB

Tabla	Partición	Orden	Índices
`siniestro-chat-dev`	`CHAT#id`	`MSG#fecha#rol` o `META`	Ninguno
`siniestro-polizas-dev`	`poliza_id`	No aplica	`by_cliente`
`siniestro-expedientes-dev`	`expediente_id`	No aplica	`by_cliente`, `by_estado`, `by_codigo`

El diseño de la tabla de chat usa una sola partición por conversación y una clave de orden que combina la marca de tiempo con el rol. Así el historial se lee con una sola consulta ordenada y no hace falta un índice. El estado de los slots se guarda en un ítem especial con clave de orden `META`, que además conserva el expediente una vez cerrado.

En la tabla de expedientes, el índice `by_estado` es el que alimenta la cola del analista y `by_codigo` resuelve la consulta pública del asegurado.

2.5. El motor de decisión

Tabla 7: Reglas de decisión y plazos

Condición	**Ruta**	**Plazo**
Falta el parte policial en un robo	Documentación pendiente	72 h
La reparación supera el 75 % del valor comercial	Pérdida total	120 h
Robo, inundación, volcadura o severidad grave	Peritaje	120 h
Costo estimado de S/. 5,000.00 a más	Peritaje	120 h
Hasta S/. 1,000.00, sin grúa y con dos piezas o menos	Pago exprés	24 h
Cualquier otro caso	Analista	72 h
Datos incoherentes entre sí	Revisión manual	48 h

Los umbrales se calibraron con la distribución del modelo de costo, no con números redondos elegidos a mano. El percentil 75 del grupo leve con dos piezas o menos está en S/. 791.00 y el percentil 75 del total de casos en S/. 5,288.00, de modo que el pago exprés corta en S/. 1,000.00 y el peritaje arranca en S/. 5,000.00. Con esos cortes, el 26 % de los casos cae en pago exprés, el 40 % en analista y el 27 % en peritaje.

La liquidación es un paso aparte del modelo. El monto que ve el asegurado es el costo estimado menos el deducible de su póliza, calculado como el mayor valor entre el 10 % del costo y un piso de S/. 350.00. Si la reparación supera el 75 % del valor comercial, el caso se resuelve como pérdida total y se indemniza el vehículo en lugar de repararlo.

2.6. Los modelos, servidos sin scikit-learn

Los dos modelos se entrenaron con scikit-learn y debían ejecutarse en un entorno serverless. El primer intento fue empaquetar la biblioteca completa: el archivo comprimido pesó 71 MB, por encima del límite de 50 MB de AWS Lambda.

La solución fue exportar los bosques a JSON y recorrerlos con la biblioteca estándar. El artefacto bajó a 2,4 MB, la predicción de costo tarda 0,2 ms y la de abandono 0,09 ms. La paridad se verificó contra scikit-learn sobre 500 casos: la diferencia máxima es de S/. 0,005 en el costo.

El detalle que casi se escapa: scikit-learn evalúa los árboles en precisión simple (`float32`). Si el runtime usa doble precisión, los empates de umbral se resuelven distinto y el monto se desvía. El código replica ese redondeo de forma explícita.

Tabla 8: Métricas de los modelos

Modelo	Casos	Métrica	Valor	Tamaño
Costo de reparación	1,000	MAE / R^2	S/. 394.16 / 0,95	6,7 MB
Abandono	1,000	Exactitud	0,93	1,4 MB

Los datos de entrenamiento son sintéticos: 1,000 siniestros con montos recalculados con precios de taller de Lima. Un parachoques delantero alternativo de Toyota Yaris cuesta alrededor de S/. 300.00 en el mercado local, y la fórmula del conjunto usa esa referencia. El rango resultante va de S/. 246.00 a S/. 17,647.00, con mediana en S/. 2,798.00.

2.7. El frontend

La interfaz tiene cuatro vistas y una decisión de diseño que atraviesa todo: el asegurado no crea una cuenta. Su identidad es la póliza, porque la aseguradora ya lo conoce y pedirle una contraseña a alguien que acaba de chocar rompe la promesa de los tres minutos.



Figura 2: Pantalla de inicio



Figura 3: La conversación: el agente saluda, escucha y pregunta lo que falta

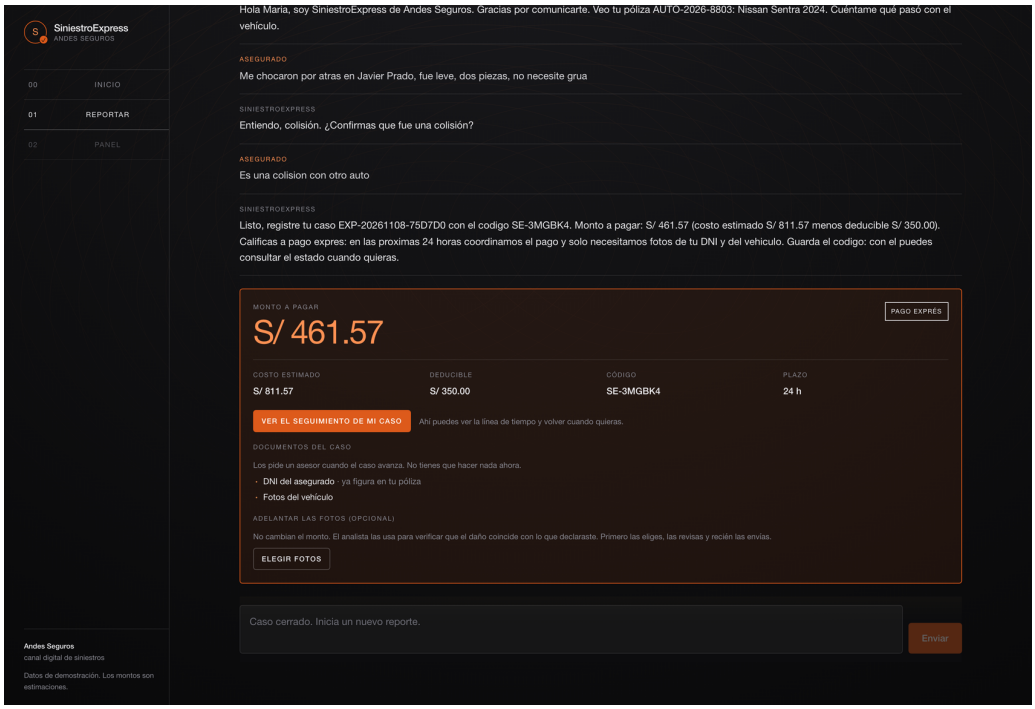


Figura 4: Veredicto del caso dentro de la conversación, con su código de seguimiento

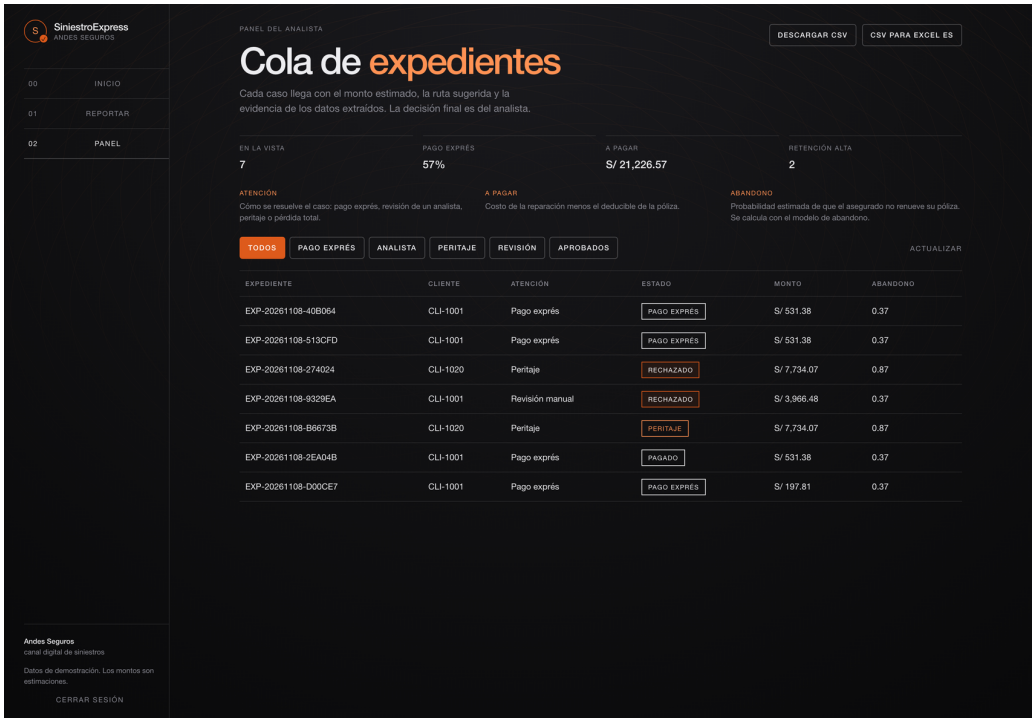


Figura 5: Panel del analista con glosario de términos



Figura 6: El veredicto en un teléfono: misma información, una sola columna

2.8. Región elegida y justificación

La región es **us-east-1** (Virginia del Norte). La decisión tiene cuatro razones:

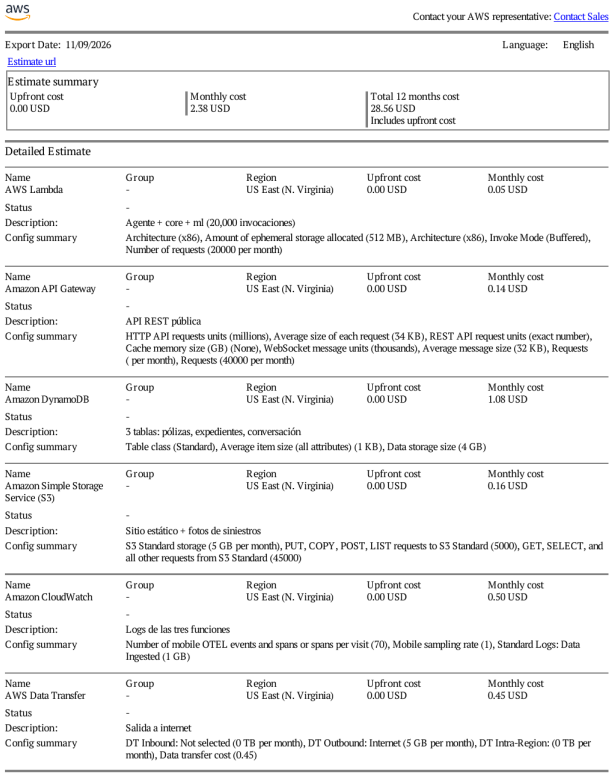
- **Disponibilidad de servicios.** Es la región donde estrenan las funcionalidades, lo que reduce el riesgo de que un servicio del proyecto no exista todavía en otra zona.
- **Latencia.** Desde Lima, la ida y vuelta ronda los 60 ms, suficiente para una conversación por turnos donde el usuario lee la respuesta antes de escribir.
- **Precio.** Es la región con las tarifas más bajas de AWS y donde aplican primero las capas gratuitas.
- **Cuenta del laboratorio.** AWS Academy habilita por defecto esa región y restringe otras, de modo que elegir otra exigiría permisos adicionales.

No se usó una región con presencia local en Perú porque no existe una en el país y porque la latencia de la conversación no es crítica: el usuario escribe a mano, no compite con un sistema de tiempo real.

2.9. Costos y plan de negocio

2.9.1. Infraestructura en AWS

La estimación se hizo en AWS Pricing Calculator con el escenario del piloto: 1,900 siniestros al mes, que generan 20,000 invocaciones de Lambda y 40,000 peticiones a API Gateway.



aws

Contact your AWS representative: Contact Sales

Export Date: 11/09/2026 Language: English

Estimate url

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
0.00 USD	2.38 USD	28.56 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
AWS Lambda	-	US East (N. Virginia)	0.00 USD	0.05 USD
Status	-			
Description:	Agente + core + ml (20,000 invocaciones)			
Config summary	Architecture (x86), Amount of ephemeral storage allocated (512 MB), Architecture (x86), Invoke Mode (Buffered), Number of requests (20000 per month)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon API Gateway	-	US East (N. Virginia)	0.00 USD	0.14 USD
Status	-			
Description:	API REST pública			
Config summary	HTTP API requests units (millions), Average size of each request (34 KB), REST API request units (exact number), Cache memory size (GB) (None), WebSocket message units (thousands), Average message size (32 KB), Requests (per month), Requests (40000 per month)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon DynamoDB	-	US East (N. Virginia)	0.00 USD	1.08 USD
Status	-			
Description:	3 tablas: pólizas, expedientes, conversación			
Config summary	Table class (Standard), Average item size (all attributes) (1 KB), Data storage size (4 GB)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon Simple Storage Service (S3)	-	US East (N. Virginia)	0.00 USD	0.16 USD
Status	-			
Description:	Sitio estático + fotos de siniestros			
Config summary	S3 Standard storage (5 GB per month), PUT, COPY, POST, LIST requests to S3 Standard (5000), GET, SELECT, and all other requests from S3 Standard (45000)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon CloudWatch	-	US East (N. Virginia)	0.00 USD	0.50 USD
Status	-			
Description:	Logs de las tres funciones			
Config summary	Number of mobile OTEL events and spans or spans per visit (70), Mobile sampling rate (1), Standard Logs: Data Ingested (1 GB)			
Name	Group	Region	Upfront cost	Monthly cost
AWS Data Transfer	-	US East (N. Virginia)	0.00 USD	0.45 USD
Status	-			
Description:	Salida a internet			
Config summary	DT Inbound: Not selected (0 TB per month), DT Outbound: Internet (5 GB per month), DT Intra-Region: (0 TB per month), Data transfer cost (0.45)			

Figura 7: Estimación en AWS Pricing Calculator

El total es de **2.38 USD al mes** y 28.56 USD al año, sin costo inicial. La mayor parte corresponde a DynamoDB (1.08 USD) y a CloudWatch (0.50 USD); Lambda queda en 0.05 USD porque casi todo el volumen cae dentro de la capa gratuita. El estimado completo está publicado en `https://calculator.aws/#/estimate?id=e06873a4658ae2ad734f853b9d80956e99e7fbcb`

2.9.2. El modelo de lenguaje

El agente usa `openai/gpt-oss-120b` a través de la API de Groq, que cobra por token procesado. Los precios publicados al cierre de este informe son:

Tabla 9: Tarifas de Groq para el modelo usado

Concepto	**Precio por millón de tokens**	**Equivalente**
Entrada (texto enviado)	0.15 USD	6.7 millones por dólar
Entrada en caché	0.075 USD	13 millones por dólar
Salida (texto generado)	0.60 USD	1.7 millones por dólar

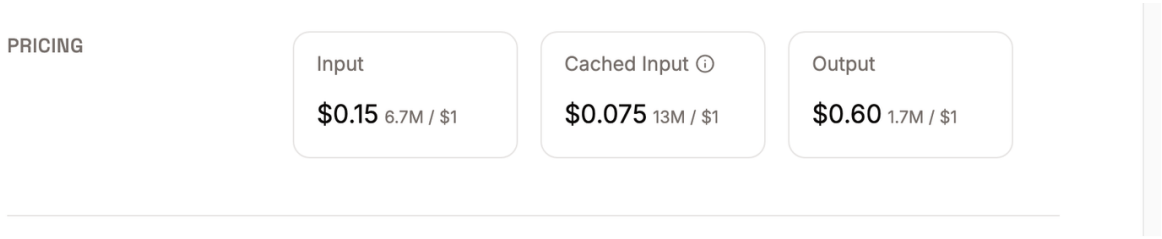


Figura 8: Tarifas publicadas en la página oficial del modelo, en Groq

Una conversación de cinco turnos consume alrededor de 8,000 tokens de entrada, porque el promedio del sistema y el historial viajan en cada turno, y unos 300 de salida. Con 1,900 casos al mes:

Tabla 10: Consumo mensual estimado de Groq

Concepto	**Tokens al mes**	**Precio unitario**	**Costo**
Entrada	15.2 millones	0.15 USD / M	2.28 USD
Salida	0.57 millones	0.60 USD / M	0.34 USD
Total			**2.62 USD**

2.9.3. El plan para Andes Seguros

Sumando las dos piezas, operar el sistema cuesta **5.00 USD al mes** para 1,900 casos: menos de tres décimos de centavo por siniestro. Sobre esa base se proponen dos suscripciones, y el precio no se fija sobre el costo de la infraestructura, que es simbólico, sino sobre el tiempo administrativo que el servicio absorbe.

Tabla 11: Suscripciones propuestas

Plan	**Precio**	**Casos al mes**	**Costo estimado**	**Margen**
Básico	20 USD	hasta 5,000	13.16 USD	6.84 USD (34 %)
Escala	100 USD	hasta 25,000	65.80 USD	34.20 USD (34 %)

El costo estimado se calculó escalando el piloto de forma lineal, que es un techo conservador: DynamoDB no crece con el número de casos sino con el almacenamiento, y Lambda y API Gateway tienen capas gratuitas que absorben el primer tramo. A 5,000 casos, Groq consume 6.90 USD y AWS unos 6.26 USD; a 25,000, la proporción se mantiene.

La lógica comercial es simple. El plan Básico cubre el doble del piloto y deja un margen del 34 %, que alcanza para operar y sostener el soporte. El plan Escala cuesta cinco veces más porque

atiende cinco veces más casos, pero el margen porcentual no cambia: el servicio no gana por volumen, gana porque reemplaza trabajo repetitivo.

El punto de equilibrio del cliente es inmediato: con un solo caso que evite media hora de trabajo administrativo, la suscripción del mes ya se pagó. Y para el proveedor también: con 1,900 casos, el costo de operación son 5.00 USD sobre una suscripción de 20.00 USD.

3. Parte C: implementación

3.1. Estructura del repositorio

El repositorio `github.com/UTEC-AII/siniestro-express` está organizado por servicio y por etapa del pipeline.

Tabla 12: Organización del repositorio

Carpeta	Contenido
`services/agente`	Chat con memoria, normalización de slots, prompt y cliente de Groq
`services/core`	Pólizas, expedientes, liquidación, estados, fotos y exportación
`services/ml`	Modelos exportados, runtime de inferencia y API de predicción
`services/ml/training`	Datos, fórmula de costos, entrenamiento y verificación de paridad
`frontend`	Aplicación React con Vite, cuatro vistas y despliegue a S3
`seed`	20 pólizas de prueba y scripts de carga y reinicio
`docs`	Arquitectura, runbook, capturas, colección de Postman y notas de dependencias
`tests`	Pruebas de integración con DynamoDB simulado

3.2. Despliegue

El procedimiento completo está en `docs/runbook-despliegue.md` del repositorio. Se ejecuta sobre la instancia de desarrollo de AWS Academy, que es una máquina Ubuntu con CloudShell, y sigue este orden:

1. **Preparar la máquina.** Node.js 20.19 o superior, Serverless Framework v4 y `boto3` para los scripts de carga. Las tres cosas se instalan una sola vez.
2. **Renovar las credenciales.** AWS Academy las entrega por cuatro horas y hay que pegarlas en `~/.aws/credentials` al empezar cada sesión.
3. **Fijar la región.** `aws configure set region us-east-1` escribe en `~/.aws/config`. Va ahí y no en el archivo de credenciales, porque ese se reescribe cada cuatro horas y se llevaría la región por delante.
4. **Clonar el repositorio** en `/home/ubuntu` y entrar a la carpeta.

5. **Definir las cinco variables** en un archivo `.env`: la clave de Groq, las tres URL de API Gateway y la del servicio core para el frontend. Los `serverless.yml` las leen al empaquetar, y si falta una el despliegue se cae con un error que no dice cuál.
6. **Desplegar en orden.** Primero `ml`, que no necesita nada y devuelve su URL; después `core`, que necesita la de `ml`; y al final `agente`, que necesita la de `core`. Cada despliegue imprime sus endpoints.
7. **Sembrar las pólizas** con `python3 seed/seed.py`. El script valida los quince campos de cada póliza antes de escribirla.
8. **Publicar el frontend** con `./scripts/deploy.sh web`. Vite incrusta las dos URL en el paquete, así que hay que compilar después de conocerlas.
9. **Comprobar.** `curl` a `/ml/health` y un `POST` a `/expedientes` confirman que los tres servicios se hablan entre sí.
10. **Redesplegar solo lo que cambió.** Si se toca el prompt, va `agente`; si cambian las reglas, `core`; si cambian los modelos, `ml`.

El script `scripts/deploy.sh` envuelve los pasos 5 y 6: verifica que las variables existan antes de llamar a Serverless, para no perder tiempo en un empaquetado que va a fallar. Las URL de cada servicio, una vez desplegados, quedan como *Output* del stack de CloudFormation, de modo que se pueden recuperar aunque se pierda la sesión.

3.3. Evidencias con Postman

La colección tiene 19 peticiones en cinco carpetas y cada una incluye pruebas automáticas. Se ejecuta de arriba hacia abajo: las variables se encadenan solas, porque cada petición guarda lo que la siguiente necesita.

La corrida completa es la evidencia más compacta del backend: las 19 peticiones se ejecutan en cadena y las **44 pruebas** pasan sin un solo fallo, en 16,7 segundos. La colección se diseñó para que cada petición deje listo lo que la siguiente necesita, de modo que una corrida limpia demuestra el flujo entero y no una suma de casos aislados.

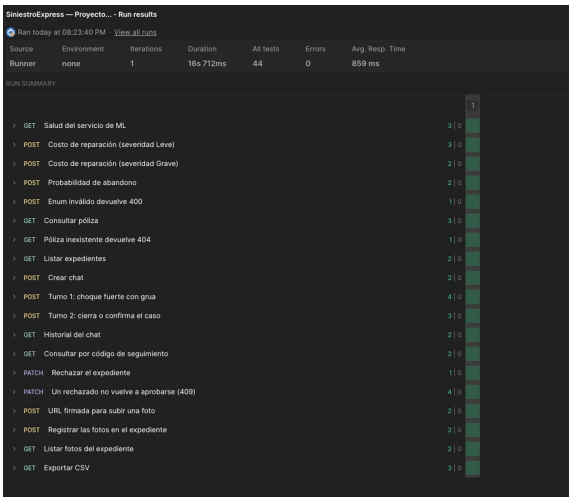


Figura 9: Corrida completa de la colección: 19 peticiones, 44 pruebas, 0 fallos

Tabla 13: Origen de las 44 pruebas de la corrida

Petición	**Pruebas**	**Qué comprueba**
Salud del servicio de ML	3	Los dos modelos cargados, 250 y 200 árboles
Costo de reparación, Leve	3	Estado, moneda y que el monto no pase el umbral de pago exprés
Costo de reparación, Grave	2	Estado y que el monto suba con la severidad
Probabilidad de abandono	2	Estado y probabilidad entre 0 y 1
Enum inválido	1	El 400 por un valor fuera del catálogo
Consultar póliza	3	Estado, vehículo y deducible de la póliza
Póliza inexistente	1	El 404 con un identificador que no existe
Listar expedientes	2	Estado y que la respuesta sea una lista
Crear chat	2	Estado 201 y la forma del identificador
Turno 1	4	Estado, piezas, grúa y tipo de siniestro
Turno 2	3	Expediente creado, formato del código y liquidación
Historial del chat	2	Estado y que la conversación se conserve
Consultar por código	2	Estado y presencia de la ruta asignada
Rechazar el expediente	1	El cambio de estado permitido
Un rechazado no se aprueba	4	El 409 y el estado actual en el cuerpo
URL firmada	2	El enlace de subida y la clave del objeto
Registrar las fotos	2	Estado y conteo de fotos registradas
Listar fotos	2	Estado y que devuelva la lista
Exportar CSV	3	Estado, tipo de contenido y separador
Total	**44**	

Las pruebas no se limitan a comprobar que la respuesta sea 200. Cada una verifica **contratos**: que el código de seguimiento tenga el formato esperado, que el monto a pagar sea menor que el costo, que la probabilidad esté entre 0 y 1, y que los tres casos que deben fallar devuelvan el código HTTP correcto con su explicación en el cuerpo.

El promedio de 859 ms por petición lo dominan las dos llamadas al modelo de lenguaje, que tardan alrededor de cinco segundos cada una; el resto responde en decenas de milisegundos. Es el comportamiento esperado cuando la latencia la fija el proveedor del modelo y no la infraestructura propia.

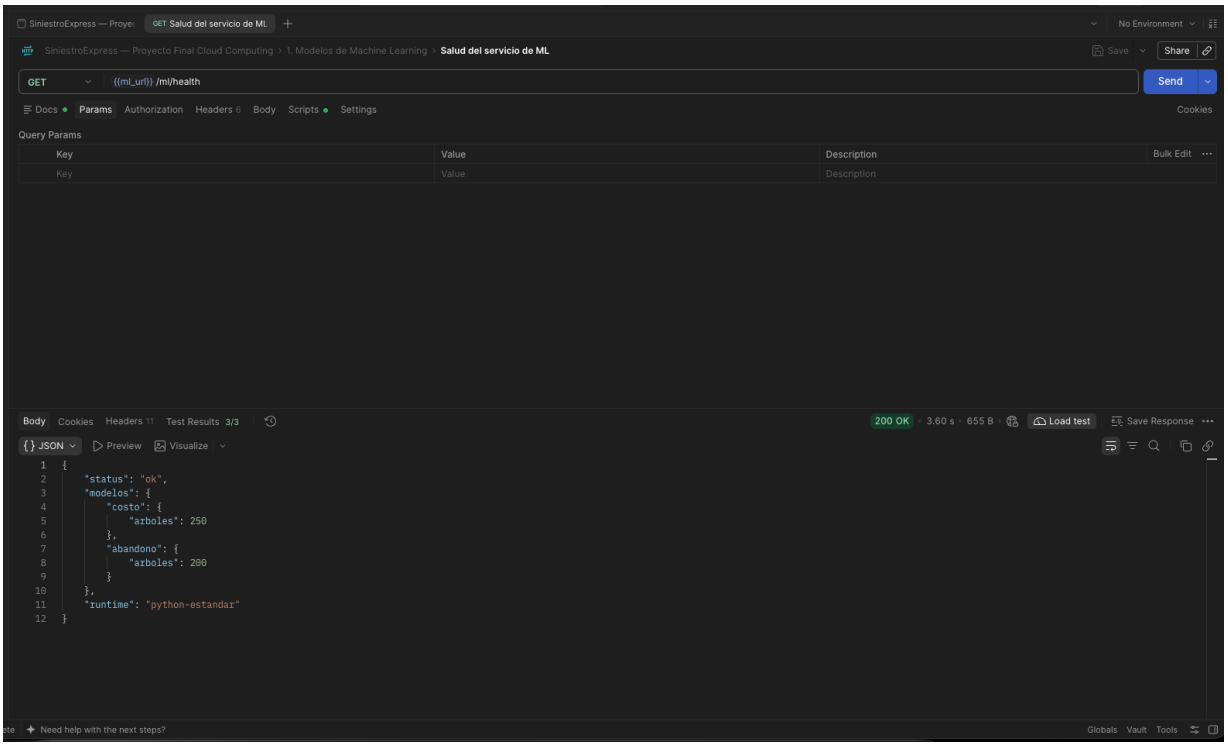


Figura 10: Colección importada y verificación del servicio de modelos

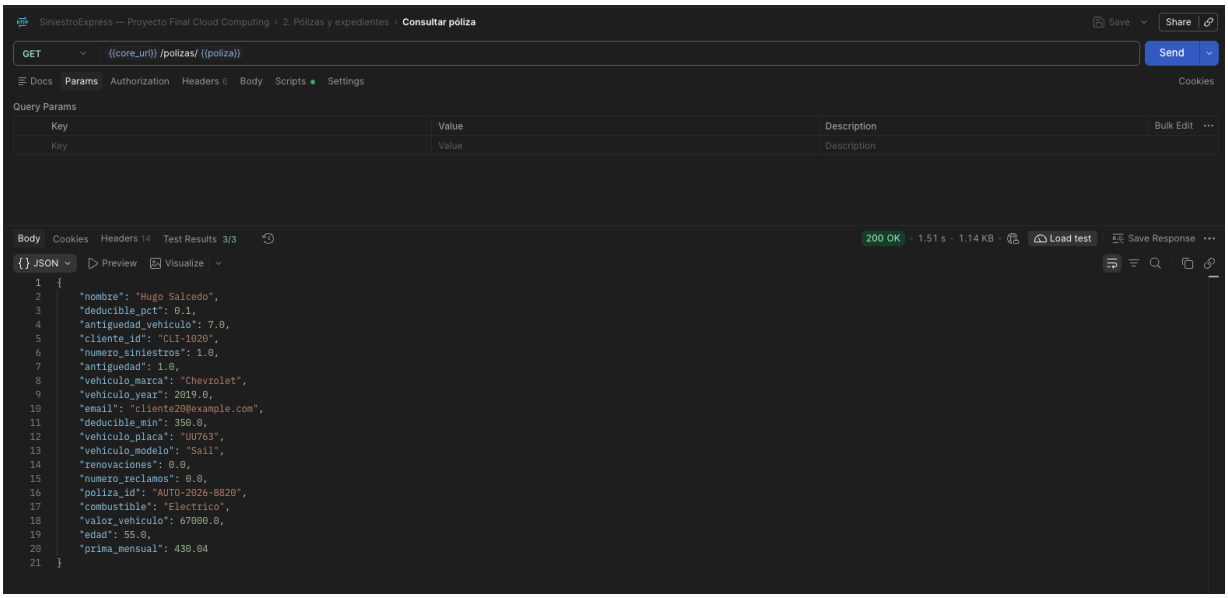


Figura 11: Consulta de póliza con vehículo y deducible

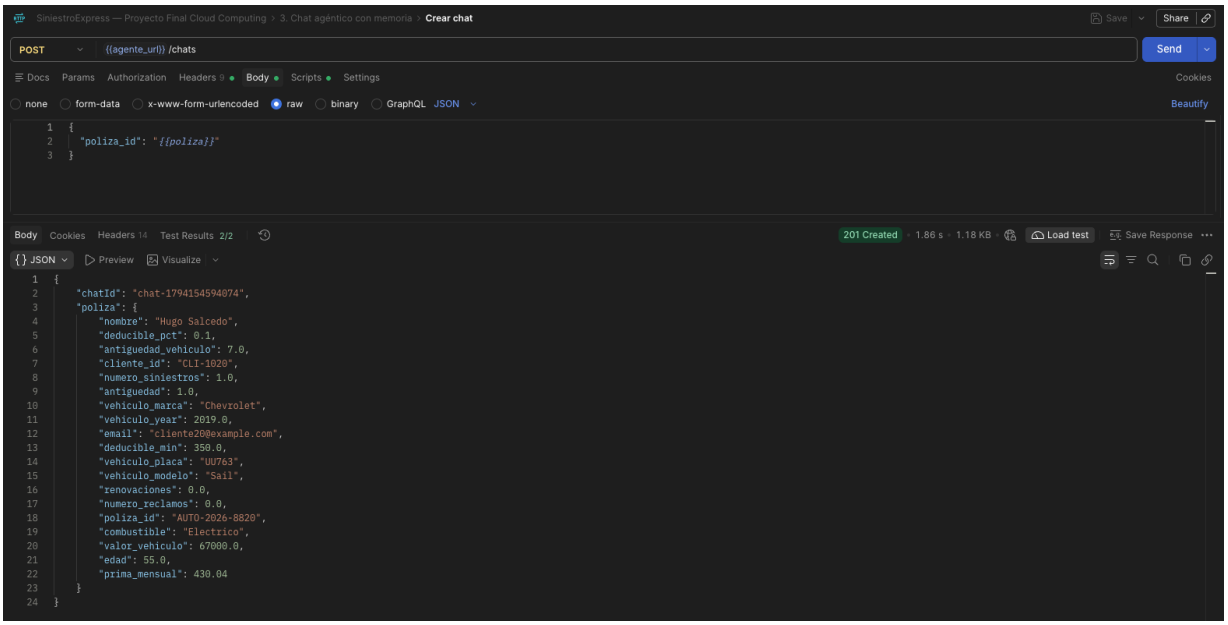


Figura 12: Creación de la conversación: el cuerpo usa la variable de la póliza

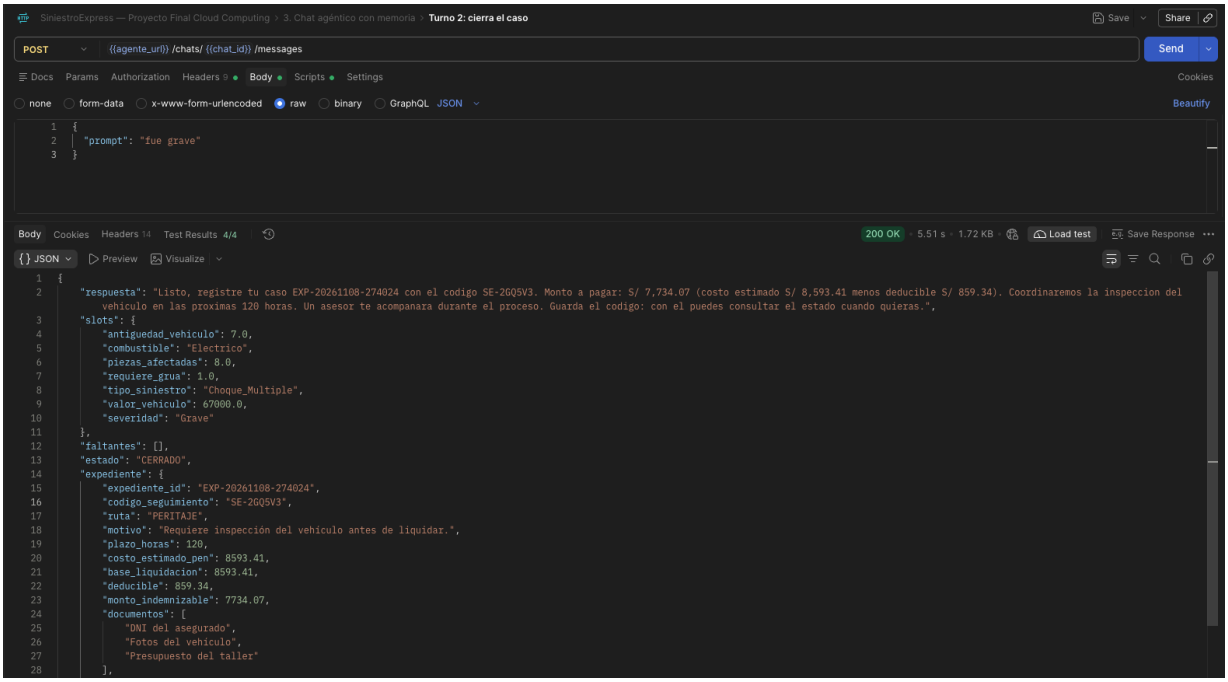


Figura 13: Segundo turno: el expediente queda creado con su código y su liquidación

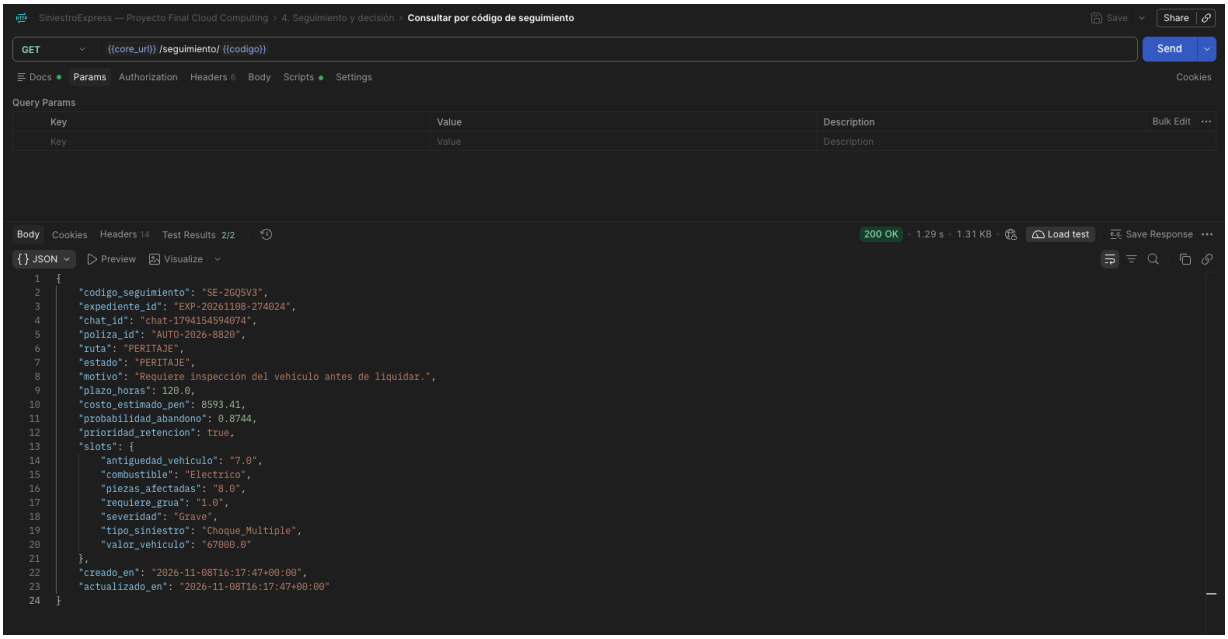


Figura 14: Consulta por código de seguimiento

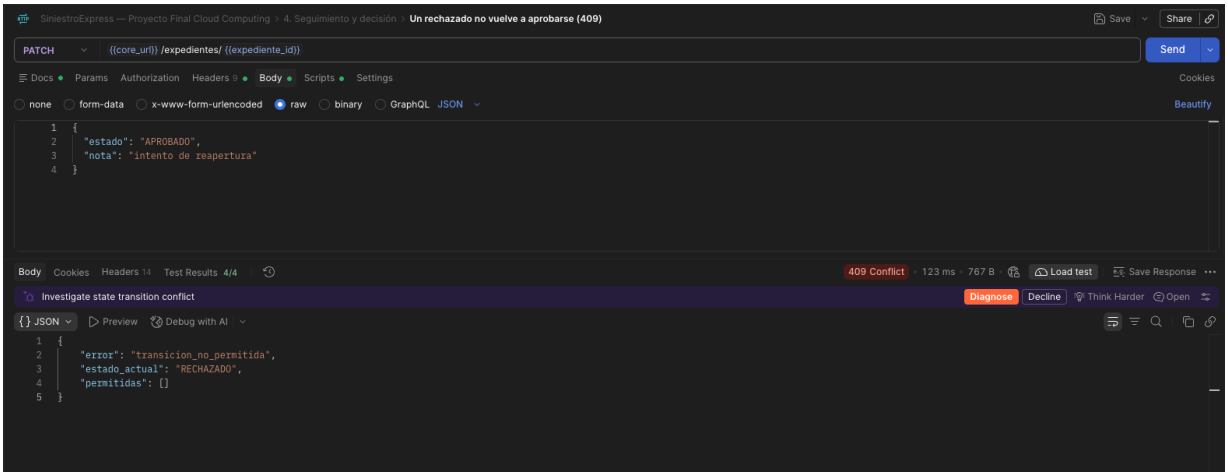


Figura 15: Rechazo de una transición no permitida

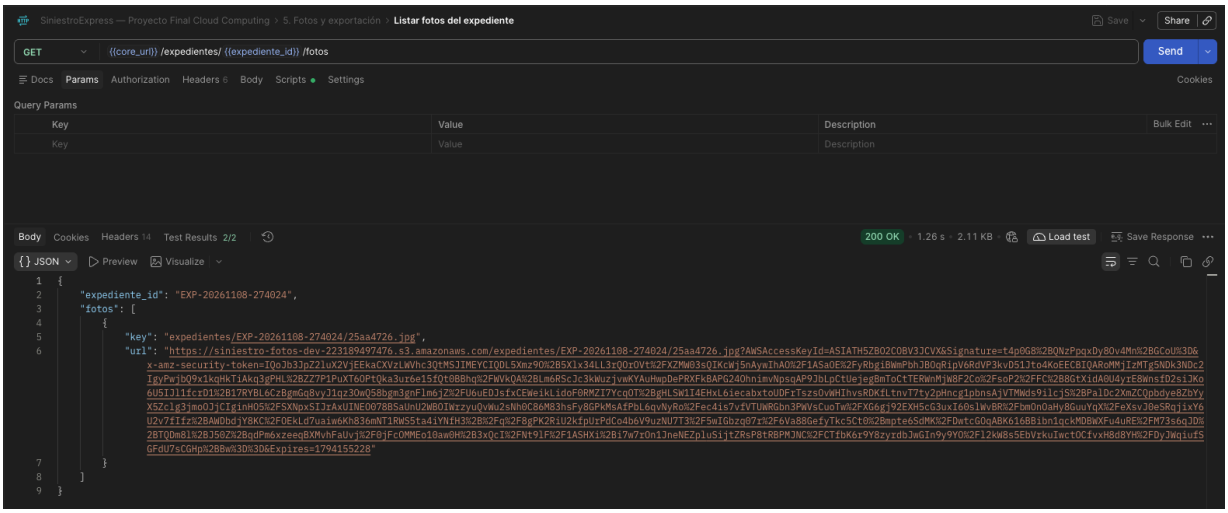


Figura 16: Listado de fotos con URL firmadas

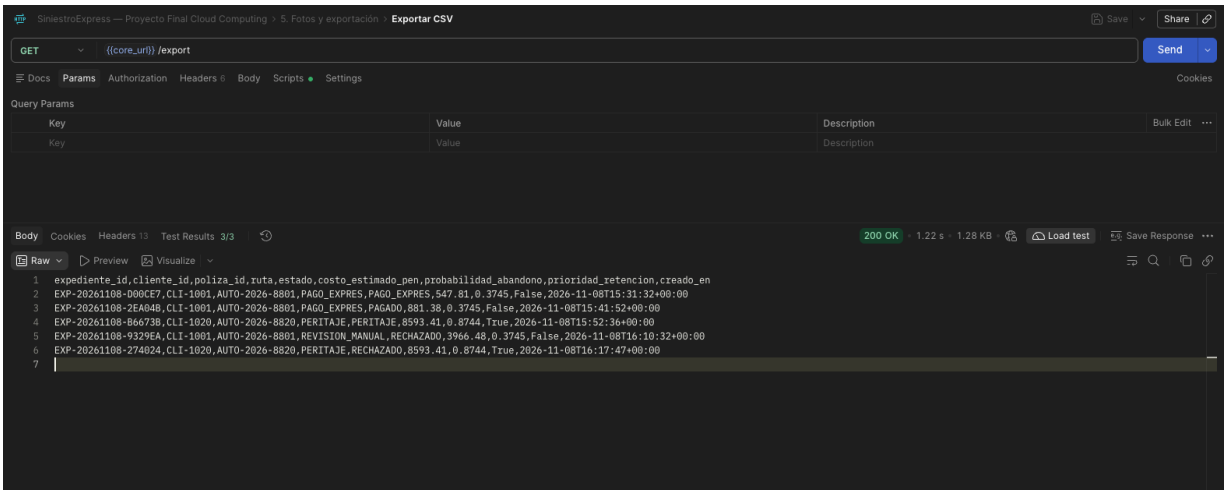


Figura 17: Exportación del CSV de expedientes

La captura de la consulta por código es la que mejor resume el sistema: en una sola respuesta aparecen el código, el expediente, la póliza, la ruta, el estado, el motivo, el plazo, el costo estimado, la probabilidad de abandono y los siete datos extraídos de la conversación.

3.4. Evidencias en la consola de AWS

El despliegue dejó los recursos con la forma descrita, y las tres consolas lo confirman. Lambda muestra las 17 funciones del proyecto en Python 3.14, empaquetadas en Zip y nombradas por servicio y etapa. DynamoDB muestra las tres tablas con sus claves de partición y el número de índices secundarios de cada una. S3 muestra los dos buckets: el del sitio web y el de las fotos.

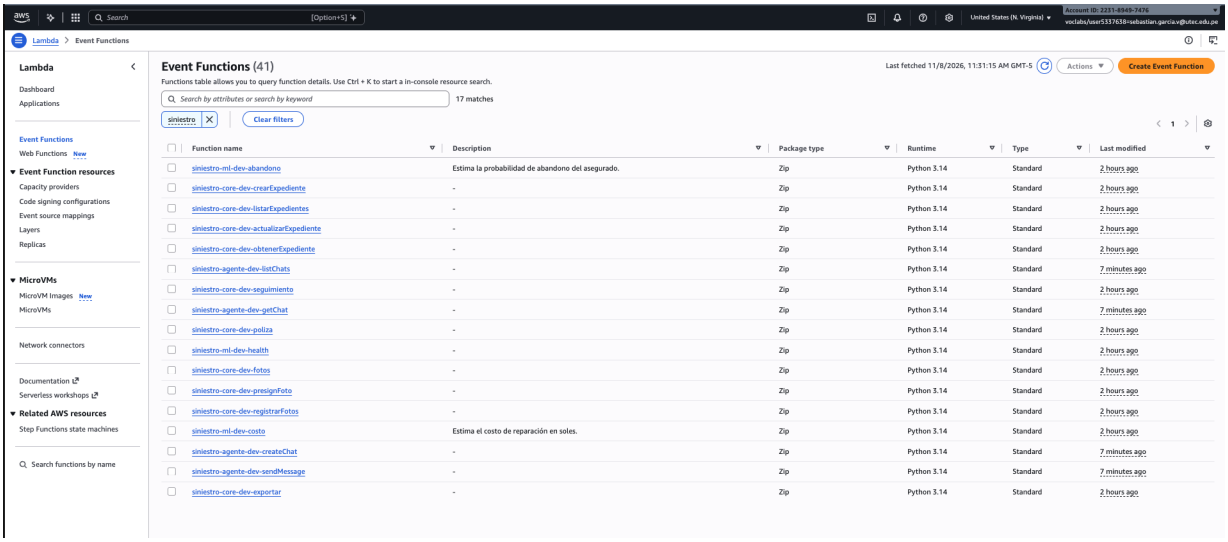


Figura 18: Las 17 funciones Lambda del proyecto

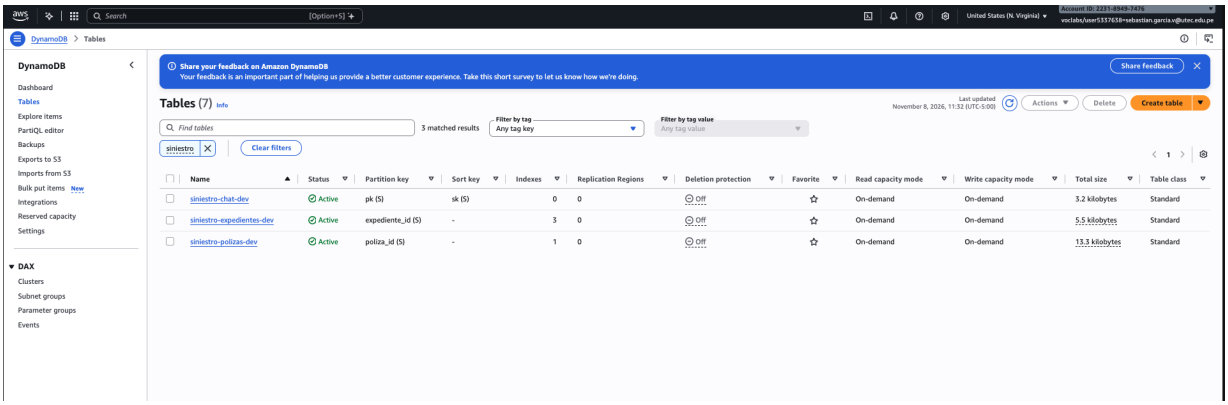


Figura 19: Las tres tablas de DynamoDB con sus índices

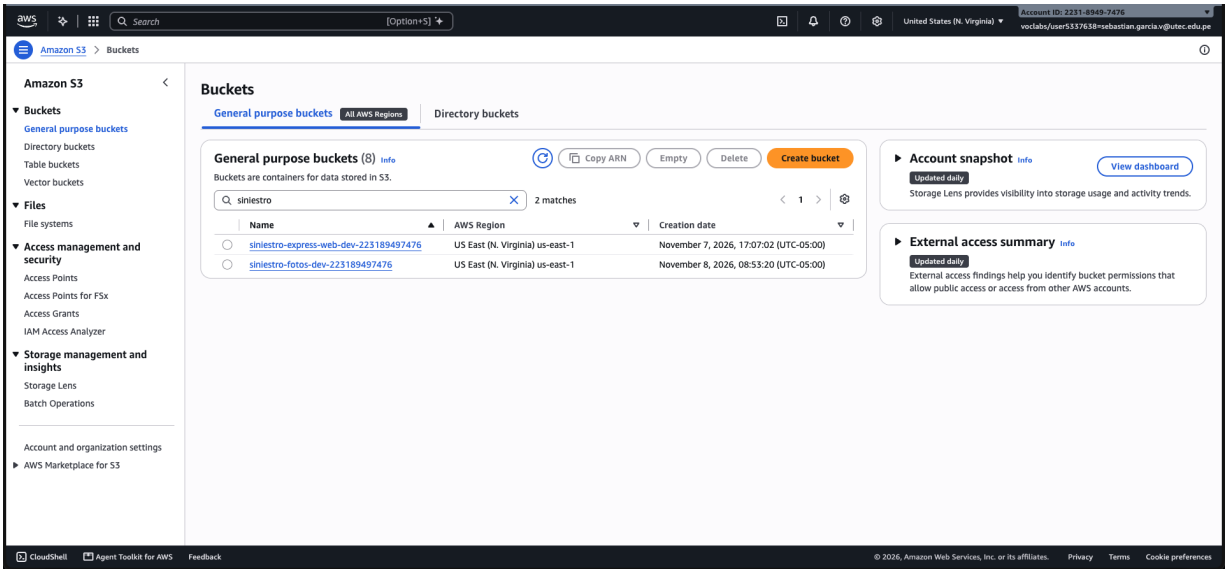


Figura 20: Los dos buckets: sitio web y fotos

3.5. Frontend conectado

El sitio publicado consume las APIs reales. No existe un modo de demostración: si las variables de entorno faltan, la aplicación muestra un aviso en pantalla en lugar de inventar respuestas. Las capturas de las figuras 2 a 6 provienen de ese despliegue.



Figura 21: Seguimiento del caso con fotos y conversación

3.6. Pruebas

Tabla 14: Pruebas automatizadas

Suite	Casos	Qué cubre
`services/ml`	4	Inferencia, validación de campos y límites de piezas
`services/core`	25	Reglas de decisión, deducible, pérdida total y máquina de estados
`services/agente`	15	Normalización de slots, alias y detección del parte policial
`tests`	6	Integración con DynamoDB simulado y flujo completo
Total	**50**	

3.7. Repositorios

- **Código.** `github.com/UTEC-AII/siniestro-express` (público).
- **Sitio publicado.** `http://siniestro-express-web-dev-223189497476.s3-website-us-east-1.amazonaws.com`
- **Colección de Postman.** `docs/postman`, dentro del mismo repositorio.

4. Conclusiones

El proyecto demuestra que un trámite de seguros puede resolverse en una conversación sin perder el control de la decisión. Esa fue la apuesta de diseño y se sostuvo de punta a punta: el modelo de lenguaje entiende el relato y redacta, pero el monto, la ruta y el plazo salen de un módulo de reglas que se lee en una pantalla, se prueba con 25 casos y se puede auditar. Un ajustador puede discutir por qué un siniestro se pagó en 24 horas y otro fue a peritaje sin abrir el código del agente.

El segundo aprendizaje es de ingeniería. Servir un modelo de machine learning en Lambda no obliga a llevar la biblioteca de entrenamiento. Exportar los bosques a JSON y recorrerlos con la biblioteca estándar bajó el paquete de 71 MB a 2,4 MB, mantuvo la latencia en 0,2 ms y no perdió precisión, con una diferencia máxima de medio céntimo sobre 500 casos. La condición para que eso funcione es respetar la precisión numérica del entrenamiento: scikit-learn recorre los árboles en `float32`, y un runtime en doble precisión cambia el monto.

El tercer aprendizaje es de producto, y no estaba en ningún requisito. El agente cerraba bien la conversación pero mostraba el costo de reparación como si fuera el monto a cobrar; el asegurado veía una cifra y el analista otra. Corregirlo fue una línea de código y el cambio más grande en claridad de todo el sistema. Lo mismo pasó con la salida del chat: sin un botón hacia el seguimiento, el asegurado quedaba en una conversación cerrada sin saber qué hacer.

En dinero, el resultado es fácil de resumir: la infraestructura cuesta 2.38 USD al mes para 1,900 casos, unos 0.00125 USD por caso, y absorbe alrededor de 1.2 horas de trabajo administrativo por siniestro. La cuenta no cierra si se presenta como un reemplazo del analista, porque no lo es; cierra cuando se lee como lo que es, un cambio en dónde se invierte el tiempo de las personas.

Lo que el proyecto no resuelve también hay que decirlo. Los modelos se entrenaron con datos sintéticos, así que los montos son coherentes entre sí pero no son precios de mercado. El panel usa un código de demostración en lugar de autenticación real. Y las fotos sirven como evidencia, no como fuente de estimación. Son tres límites conocidos, no tres sorpresas.

Trabajo futuro

- Reentrenar con el historial real y recalibrar los umbrales.
- Autenticación con Cognito y roles para el analista.
- Detección de daño desde la foto, hoy solo evidencia.
- Asignación de taller por disponibilidad y zona.